

Arquitectura de Solución en la Nube (Microsoft Azure)

En el documento se detalla la estrategia de arquitectura en la nube para el sistema de estimación y proyección de costos de equipos de construcción, con el fin de equilibrar la agilidad del desarrollo con los estándares de nivel empresarial, la solución se estructuró en dos etapas estratégicas:

1. una **Arquitectura Objetivo (Producción)**, diseñada para operar a gran escala de forma automatizada, segura y gobernada bajo prácticas de MLOps.
 2. una **Arquitectura de Demostración (Demo)**, diseñada para validación rápida y bajo costo.
-

1. Estrategia de Arquitectura

Para garantizar el éxito del proyecto, se adoptó un enfoque de entrega evolutivo:

¿Por qué la arquitectura actual es perfecta para la Demo?

La prioridad de la demo es **validar la hipótesis de negocio y la viabilidad técnica** (demostrar que el agente de IA, la API y la aplicación web interactúan correctamente con los modelos estadísticos de `cost_forecast`).

Para esto, se optó por un enfoque **Serverless e Inmutable**: todo el pipeline de datos y los artefactos analíticos precalculados se empaquetaron directamente dentro de contenedores Docker desplegados en **Azure Container Apps**.

- **Ventajas clave:** Reducción de costos a cero cuando no se usa (gracias a la escala a 0 réplicas), despliegue en minutos, nula sobrecarga de mantenimiento de infraestructura y total independencia para iterar el código.

¿Por qué se propone una arquitectura diferente para Producción?

En un entorno productivo real, las condiciones cambian. Los precios de las materias primas se actualizan diariamente, los modelos pierden precisión por el cambio del entorno (*data drift*) y múltiples áreas de la empresa podrían consumir la API en paralelo.

La arquitectura objetivo pasa de un enfoque "estático e integrado" a uno **desacoplado y gobernado**. Se introduce orquestación automatizada para la ingesta de datos frescos, almacenamiento estructurado por capas (Data Lakehouse), ciclos de vida de modelos controlados (MLflow/Azure ML) y seguridad perimetral de nivel empresarial.

2. Arquitectura Objetivo (Producción)

Esta arquitectura está diseñada para automatizar el ciclo de vida completo del dato y del modelo (MLOps), garantizando alta disponibilidad, seguridad y resiliencia.

Diagrama de la Arquitectura Propuesta

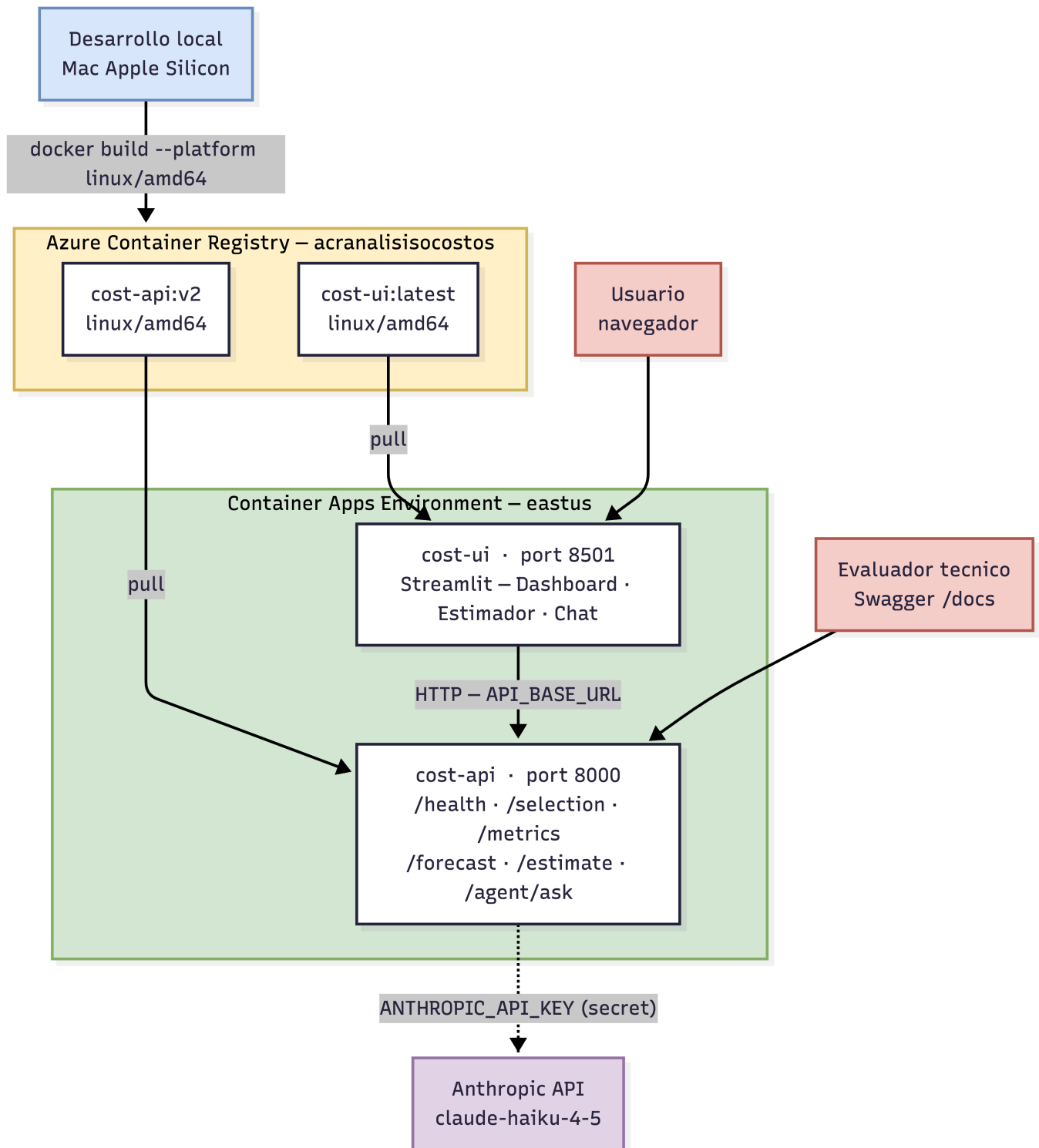
2 / 6

Capa	Servicio Azure	Justificación y Rol en la Solución
Almacenamiento	Azure Data Lake Storage Gen2 (ADLS)	Organiza la información bajo el patrón <i>Medallion</i> : • Bronze: Archivos CSV crudos en sus formatos heterogéneos originales. • Silver: Datos normalizados, parseados y agregados mensualmente. • Gold: Artefactos finales, métricas de negocio y proyecciones listas para consumo.
Capa de Cómputo y ML	Azure Databricks o Azure Machine Learning	Ejecuta de forma distribuida el paquete modular <code>cost_forecast</code> y se encargaría del análisis de estacionariedad, la votación por consenso para selección de variables y la simulación de Monte Carlo para la propagación de incertidumbre.
Gobernanza de Modelos	Azure ML Model Registry	Se encargaría de registrar, versionar y auditar los modelos seleccionados (SARIMAX y ElasticNet), almacenando su historial de métricas (<i>MAPE</i> , <i>RMSE</i>) y permitiendo realizar <i>rollbacks</i> inmediatos o despliegues controlados.
Servicios de Aplicación	Azure Container Apps	Hospeda de forma elástica y desacoplada tanto la API REST (FastAPI) como la Interfaz Web (Streamlit) , asegurando que el consumo financiero o técnico no interfiera con los procesos de cómputo pesado.
Capa del Agente de IA	Azure Container Apps + Anthropic API	Aloja el agente conversacional construido con LangGraph . El agente tiene la capacidad de consultar de forma segura los datos en la capa <i>Gold</i> y complementar sus respuestas con búsquedas web en tiempo real para dar contexto de mercado.
Seguridad y Secretos	Azure Key Vault	Centraliza la gestión de claves y credenciales (como la <code>ANTHROPIC_API_KEY</code>), eliminando por completo cualquier variable de entorno expuesta en el código fuente.
Identidad y Acceso	Microsoft Entra ID	Gobierna la seguridad mediante el principio de menor privilegio, utilizando identidades administradas (<i>Managed Identities</i>) para que los servicios se comuniquen entre sí sin requerir contraseñas explícitas.
Observabilidad	Azure Monitor + Application Insights	Monitorea la salud de la API, tiempos de respuesta del agente y, críticamente, vigila la degradación del modelo (<i>Model Drift</i>). Si el <i>MAPE</i> supera el umbral tolerado, envía una alerta o dispara un reentrenamiento automático.

3. Demo Implementada en Azure

El prototipo funcional de la solución se encuentra desplegado y operativo bajo un esquema eficiente en costos y recursos.

Diagrama del Demo Desplegado



Recursos Desplegados

Se configuró un entorno Serverless en la región **eastus**, bajo el grupo de recursos **rg-analisis-costos**:

1. **Azure Container Registry (**acranalisisocostos.azurecr.io**)**: Repositorio privado donde se gestionan y almacenan las imágenes Docker optimizadas para arquitectura x64.
2. **Container App – API (**cost-api...azurecontainerapps.io**)**: Microservicio basado en FastAPI que expone los endpoints de predicción, simulación de escenarios y el motor del agente, el mismo tiene una política de escalado de **0 a 2 réplicas** para eliminar costos en tiempos de inactividad.

3. **Container App — UI** (**cost-ui...azurecontainerapps.io**): Interfaz gráfica en Streamlit para el usuario final, conectada de manera segura a la URL interna de la API.

4. Brecha Tecnológica entre Demo y Producción

Para entender cómo evolucionará este prototipo a la plataforma empresarial, se presenta la siguiente matriz de diferencias:

Característica	Solución en Demo (Actual)	Solución en Producción (Objetivo)
Gestión de Datos	Estática e inmutable (datos e históricos quemados dentro de la imagen Docker).	Dinámica (Lectura y escritura directa sobre capas Bronze/Silver/Gold en ADLS Gen2).
Ciclo de Vida de ML	Local y manual. Los artefactos se generan al construir el contenedor (make all).	Automatizado. Entrenamiento distribuido en Azure ML / Databricks con registro formal en Model Registry .
Ingesta y Flujos	No requiere orquestador, se asume un histórico fijo para la validación.	Pipelines programados y orientados a eventos gestionados por Azure Data Factory .
Gestión de Secretos	Inyectados como variables de entorno seguras en el despliegue del contenedor.	Extracción dinámica en tiempo de ejecución desde Azure Key Vault mediante identidades asignadas por software.
Monitoreo y Drift	Logs básicos de consola a través de la salida estándar del contenedor.	Telemetría avanzada y alertas de negocio integradas en Application Insights .

5. Guía de Despliegue del Demo (Pasos Reales)

A continuación, se documentan los comandos de la CLI de Azure (**az**) ejecutados para la construcción y aprovisionamiento del entorno actual:

```
#### Variables de Entorno del Proyecto
RESOURCE_GROUP=rg-analisis-costos
LOCATION=eastus
ACR_NAME=acranalisisocostos
ENV_NAME=env-analisis-costos

#### 1. Inicialización de proveedores en la suscripción
az provider register -n Microsoft.OperationalInsights --wait

#### 2. Creación y obtención de credenciales del Registro de Contenedores (ACR)
az acr create --resource-group $RESOURCE_GROUP --name $ACR_NAME --sku Basic --admin-enabled true
ACR_USER=$(az acr credential show -n $ACR_NAME --query username -o tsv)
ACR_PASS=$(az acr credential show -n $ACR_NAME --query "passwords[0].value"
```

```
-o tsv)
```

```
#### 3. Construcción multiplataforma (Garantiza compatibilidad de
arquitecturas ARM/Mac a x64/Cloud)
```

```
docker build --platform linux/amd64 -f Dockerfile.api -t
$ACR_NAME.azurecr.io/cost-api:v2 .
docker build --platform linux/amd64 -f Dockerfile.ui -t
$ACR_NAME.azurecr.io/cost-ui:latest .
```

```
#### 4. Carga de imágenes al entorno de Azure
```

```
docker push $ACR_NAME.azurecr.io/cost-api:v2
docker push $ACR_NAME.azurecr.io/cost-ui:latest
```

```
#### 5. Creación del entorno base para Container Apps
```

```
az containerapp env create --name $ENV_NAME --resource-group
$RESOURCE_GROUP --location $LOCATION
```

```
#### 6. Despliegue del Microservicio (API REST + Agente)
```

```
az containerapp create --name cost-api --resource-group $RESOURCE_GROUP \
--environment $ENV_NAME --image $ACR_NAME.azurecr.io/cost-api:v2 \
--registry-server $ACR_NAME.azurecr.io \
--registry-username $ACR_USER --registry-password $ACR_PASS \
--target-port 8000 --ingress external \
--secrets anthropic-key=<ANTHROPIC_API_KEY> \
--env-vars ANTHROPIC_API_KEY=secretref:anthropic-key \
--min-replicas 0 --max-replicas 2 --cpu 0.5 --memory 1.0Gi
```

```
#### 7. Despliegue de la Interfaz de Usuario (Streamlit Web App)
```

```
API_URL=cost-api.redsky-57888008.eastus.azurecontainerapps.io
az containerapp create --name cost-ui --resource-group $RESOURCE_GROUP \
--environment $ENV_NAME --image $ACR_NAME.azurecr.io/cost-ui:latest \
--registry-server $ACR_NAME.azurecr.io \
--registry-username $ACR_USER --registry-password $ACR_PASS \
--target-port 8501 --ingress external \
--env-vars API_BASE_URL=https://$API_URL \
--secrets anthropic-key=<ANTHROPIC_API_KEY> \
--env-vars ANTHROPIC_API_KEY=secretref:anthropic-key \
--min-replicas 0 --max-replicas 1 --cpu 0.5 --memory 1.0Gi
```